コンピュータサイエンス第一 第三回課題解答解説

今回のポイント

- リスト(list)の添字アクセス a[i] と len()
- for i in range(...) による添字を使った繰り返し
- 「現在の最大値」「合計」のように、ループで値を更新していく変数
- 辞書(dict)によるカウント、文字列メソッド isalpha() / lower()

各問の解答解説

問 1

```
def maximum(input_list):
    max_value = input_list[0]
    for i in range(1, len(input_list)):
        if input_list[i] > max_value:
            max_value = input_list[i]
    return max_value
```

これによって､リストの中で一番大きい値が返ってきます｡例えば maximum([1,2,3,4,5,99,3,5,4,2,1,5]) は 99、maximum([10]) は 10 です。プログラムの流れとしては、まず max_value に先頭の要素 input_list[0] を入れておき、「ここまでで一番大きい値」とします。次に for i in range(1, len(input_list)) で、i に 1 から最後の添字までを順番に入れて繰り返します。各回で input_list[i] > max_value、つまり今見ている要素がこれまでの最大値より大きいかを調べ、大きければ max_value をその値に更新します。最後まで見終わったら、max_value が最大値になっているので return max_value で返します。

細かい解説

range(1, len(input_list)) が 0 ではなく 1 から始まるのは、0 番目はすでに max_value に入れてあるからです。[10] のように要素が 1 つだけのときは range(1, 1) となり、ループは 1 回も実行されずに先頭の 10 がそのまま返ります。return max_value は for の外(インデントが一段浅い位置)に書きます。ループの中に書くと、1 回目の比較だけで関数が終わってしまいます。[1,2,10,2,1,10,9] のように最大値が複数あっても、>(より大きい)の比較なので問題なく 10 が返ります。max_value の初期値を 0 にするのは避けましょう。リストが負の数だけのとき(例えば [-5,-3])に、実際には存在しない 0 が返ってしまいます。

注. この問題では組み込み関数 `max` の利用は禁止されています。同様に、`input_list.sort()` のように並べ替えて末尾を取る方法も、この課題の趣旨(ループの練習)からは外れるので避けましょう。

問 2

```
def dot_product(a, b):
    total = 0
    for i in range(len(a)):
        total += a[i] * b[i]
    return total
```

これによって、2 つのベクトルの内積が返ってきます。例えば `dot_product([1,2,3],[4,5,6])` は `1*4 + 2*5 + 3*6` で 32 になります。プログラムの流れとしては、まず合計を入れる変数 `total` を `0` で用意します。次に `for i in range(len(a))` で、`i` に 0 から `len(a)-1` までを順番に入れて繰り返します。各回で `a[i] * b[i]`、つまり同じ位置の要素同士の積を `total` に足していきます。最後に `return total` で合計を返します。

細かい解説

問題文の式 $\sum_{i=1}^{n} a_i b_i$ は「$a_i b_i$ を i について全部足す」という意味で、そのままコードに対応しています。数式は 1 始まり、Python の添字は 0 始まりなので、`range(len(a))` で 0 から数えればちょうど n 個の要素を回れます。`total += a[i] * b[i]` は `total = total + a[i] * b[i]` の略記です。合計の初期値は `0` にします。足し算の「何も足していない状態」が `0` だからです(掛け算で積を求めるときは初期値を `1` にします)。`a` と `b` の長さは等しいと保証されているので、`len(a)` だけで回して大丈夫です。

注. `total` を `for` の中で `0` にしてしまうと、毎回リセットされて最後の 1 項だけが残ってしまいます。初期化は必ずループの前に書きましょう。

問 3

```
def frequency(input_text):
    counts = {}
    for ch in input_text:
        if ch.isalpha():
            c = ch.lower()
            if c in counts:
                counts[c] += 1
```

```
            else:
                counts[c] = 1
    return counts
```

これによって、各アルファベットの出現回数を表す辞書が返ってきます。例えば `frequency("Babbba BA ba!")` は `{'b': 6, 'a': 4}` です（順番は問われません）。プログラムの流れとしては、まず空の辞書 `counts` を用意します。次に `for ch in input_text` で、文字列から1文字ずつ `ch` に取り出して繰り返します。`ch.isalpha()` が True のとき（アルファベットのとき）だけ処理を行い、`!` や空白は無視されます。アルファベットだった場合は `ch.lower()` で小文字にして `c` とし、大文字と小文字を同じ文字として扱えるようにします。すでに `c` が辞書にあれば `counts[c] += 1` で1増やし、なければ `counts[c] = 1` で新しく登録します。最後に `return counts` で辞書を返します。

細かい解説

`frequency("AaBb")` の中身を追ってみましょう。`counts` は次のように変化します。

```
最初          counts={}
'A' → 'a'     counts={'a':1}
'a'           counts={'a':2}
'B' → 'b'     counts={'a':2, 'b':1}
'b'           counts={'a':2, 'b':2}
```

辞書に存在しないキーを `counts[c] += 1` で読むとエラー（`KeyError`）になります。そのため、`if c in counts:` で「すでにあるか」を先に確認して場合分けしています。`""` や `"!@#$%"` のようにアルファベットが1つもない入力では、何も登録されず空の辞書 `{}` が返ります。テスト表の通りですね。`isalpha()` は日本語のひらがなども True にします。この課題では英字を前提としているので問題になりませんが、「26文字のアルファベット」に限定したいときは別の判定が必要になります。

注. `return counts` は `for` の外に書きます。ループの中に書くと、最初の1文字を処理しただけで関数が終わってしまいます。